09/10/2026

# Genetic Algorithm for optimization

pseudocode:

```
FUNCTION Fitness(x) returns f(x)

FUNCTION GeneticAlgorithm
    Initialize population randomly
    While stopping condition is false:
        Evaluate fitness(x)
        select parents
        apply crossover to parents
        apply mutation to offsprings
        Replace old population with new
    END of while
    returns best solution

END of function

Call GeneticAlgorithm().
```

## Inventory & Warehouse Optimization

problem:- Optimize stock replenishment and warehouse item allocation.

obj:- Minimise inventory and storage cost while meeting demand and satisfying the warehouse constraints.

pseudocode for given problem statement:-

```
Function fitness(solution)
    calculate Total Inventory cost
    calculate total storage cost
    Calculate penalty          // for unmet demand
    RETURN Total cost + penalty
END Function
```

```
FUNCTION InitializePopulation()
    Generate random replenishment quantities
    Assign stock to warehouse
    RETURN population
END FUNCTION

FUNCTION Selection(population)
    Select the best parent solutions
    RETURN parents
END FUNCTION

FUNCTION Crossover(parents)
    Combine parent solutions
    RETURN offsprings
END FUNCTION

FUNCTION Mutation(offspring)
    Randomly change stock quantities
    Change warehouse locations
    RETURN offspring
END FUNCTION

FUNCTION GeneticAlgorithm()
    population = InitializePopulation()
    while stopping condition is not met
        Evaluate fitness for each solution
        parents = selection(population)
        offspring = crossover(parents)
        offspring = Mutation(offspring)
        Repairs solutions to satisfy constraints
        population = Replace(population, offspring)
end of while
```

returns best solution
end of FUNCTION

best solution = Genetic Algorithm()
Display replenishment quantities
Display Warehouse allocation
Display minimum total cost

output:-

Enter customer demand: 100
Enter current stock: 20
Enter number of warehouses: 2

Warehouse 1
Enter warehouse capacity: 60
Enter storage cost per unit: 2

Warehouse 2
Enter warehouse capacity: 80
Enter storage unit cost: 3
Enter replenishment cost per unit: 5

---Optimization results---
Customer demand: 100
Current stock: 20
Warehouse 1 replenishment: 57 capacity: 60
Warehouse 2 replenishment: 23 capacity: 80
Total available stock: 100
Demand status: fully met
Minimum Total cost: 583

```
import random

# User input
demand = int(input("Enter customer demand: "))
current_stock = int(input("Enter current stock: "))

n = int(input("Enter number of warehouses: "))

warehouse_capacity = []
storage_cost = []

for i in range(n):
    print("\nWarehouse", i + 1)
    warehouse_capacity.append(
        int(input("Enter warehouse capacity: "))
    )
    storage_cost.append(
        int(input("Enter storage cost per unit: "))
    )

replenishment_cost = int(input("Enter replenishment cost per unit: "))

POPULATION_SIZE = 10
GENERATIONS = 50
MUTATION_RATE = 0.1


# Generate a random solution
def create_solution():
    return [
        random.randint(0, cap)
        for cap in warehouse_capacity
    ]


# Calculate total cost
def fitness(solution):
    total_stock = current_stock + sum(solution)
    shortage = max(0, demand - total_stock)

    cost = sum(
        solution[i] * (replenishment_cost + storage_cost[i])
        for i in range(n)
    )

    cost += shortage * 1000
    return cost


# Select the best solution from random candidates
def selection(population):
    candidates = random.sample(
        population, min(3, len(population))
    )
    return min(candidates, key=fitness)


# Crossover
def crossover(parent1, parent2):
    point = random.randint(1, n) if n > 1 else 1

    child = (
        parent1[:point] + parent2[point:]
    )
    return child


# Mutation
def mutation(solution):
    if random.random() < MUTATION_RATE:
        i = random.randint(0, n - 1)
        solution[i] = random.randint(
            0, warehouse_capacity[i]
        )
    return solution


# Genetic Algorithm
def genetic_algorithm():
    population = [
        create_solution()
```

```
        create_solution()
        for _ in range(POPULATION_SIZE)
    ]

    for _ in range(GENERATIONS):
        new_population = []

        for _ in range(POPULATION_SIZE):
            parent1 = selection(population)
            parent2 = selection(population)

            child = crossover(parent1, parent2)
            child = mutation(child)

            new_population.append(child)

        population = new_population

    return min(population, key=fitness)


# Main program
best = genetic_algorithm()

print("\n--- Optimization Results ---")
print("Customer Demand:", demand)
print("Current Stock:", current_stock)

for i in range(n):
    print(
        "Warehouse", i + 1,
        "Replenishment:", best[i],
        "Capacity:", warehouse_capacity[i]
    )

print("Total Available Stock:", current_stock + sum(best))

if current_stock + sum(best) >= demand:
    print("Demand Status: Fully Met")
else:
    print("Demand Status: Not Fully Met")

print("Minimum Total Cost:", fitness(best))
```

```
Enter customer demand: 100
Enter current stock: 20
Enter number of warehouses: 2

Warehouse 1
Enter warehouse capacity: 60
Enter storage cost per unit: 2

Warehouse 2
Enter warehouse capacity: 80
Enter storage cost per unit: 3
Enter replenishment cost per unit: 5

--- Optimization Results ---
Customer Demand: 100
Current Stock: 20
Warehouse 1 Replenishment: 57 Capacity: 60
Warehouse 2 Replenishment: 23 Capacity: 80
Total Available Stock: 100
Demand Status: Fully Met
Minimum Total Cost: 583
```